



Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №3

Вариант 666
по дисциплине

«Распределённые системы хранения данных»:

Выполнили: Ястребов-Амирханов Алекси, Исупов Никита Александрович
Группа: Р3332/Р3331
Преподаватель: Афанасьев Дмитрий Борисович

Санкт-Петербург,

2026

Оглавление

Задание	3
Этап 1. Конфигурация.	4
1. Создание и подготовка виртуальных машин.....	4
2. Организация пространства для скриптов.....	4
2.1. Файл env (общие переменные окружения)	4
2.2. Файл common (вспомогательные функции).....	5
3. Настройка PostgreSQL (поточная репликация)	6
3.1. На узле primary	6
3.2. На узле standby	7
3.3. На узле client.....	8
3.4. Проверка репликации.....	8
3.5. Демонстрация клиентских сессий	8
Этап 2. Симуляция и обработка сбоя.	9
1. Заполнение дискового пространства на primary.....	9
2. Попытка записи данных на primary (клиент)	10
3. Переключение (failover) на standby	11
4. Подготовка standby для будущего восстановления primary	12
5. Восстановление основного узла.....	12
5.1. Восстановление старого primary как реплики.....	12
5.2. Проверка статуса репликации.....	14
5.3. Проверка целостности данных.....	14
Вывод	15

Задание.

Цель работы - ознакомиться с методами и средствами построения отказоустойчивых решений на базе СУБД Postgres; получить практические навыки восстановления работы системы после отказа.

Работа рассчитана на двух человек и выполняется в три этапа: настройка, симуляция и обработка сбоя, восстановление.

Требования к выполнению работы

- В качестве хостов использовать одинаковые виртуальные машины.
- В первую очередь необходимо обеспечить сетевую связность между ВМ.
- Для подключения к СУБД (например, через `psql`), использовать отдельную виртуальную или физическую машину.
- Демонстрировать наполнение базы и доступ на запись на примере **не менее, чем двух** таблиц, столбцов, строк, транзакций и клиентских сессий.

Этап 1. Конфигурация

Развернуть postgres на двух узлах в режиме потоковой репликации. Не использовать дополнительные пакеты. Продемонстрировать доступ в режиме чтение/запись на основном сервере. Продемонстрировать, что новые данные синхронизируются на резервный сервер.

Этап 2. Симуляция и обработка сбоя

2.1 Подготовка:

- Установить несколько клиентских подключений к СУБД.
- Продемонстрировать состояние данных и работу клиентов в режиме чтение/запись.

2.2 Сбой:

Симулировать переполнение дискового пространства на основном узле - заполнить всё свободное пространство раздела с PGDATA “мусорными” файлами.

2.3 Обработка:

- Найти и продемонстрировать в логах релевантные сообщения об ошибках.
- Выполнить переключение (failover) на резервный сервер.
- Продемонстрировать состояние данных и работу клиентов в режиме чтение/запись.

Восстановление

- Восстановить работу основного узла - откатить действие, выполненное с виртуальной машиной на этапе 2.2.
- Актуализировать состояние базы на основном узле - накатить все изменения данных, выполненные на этапе 2.3.
- Восстановить исправную работу узлов в исходной конфигурации (в соответствии с этапом 1).
- Продемонстрировать состояние данных и работу клиентов в режиме чтение/запись.

Этап 1. Конфигурация.

1. Создание и подготовка виртуальных машин

```
# Создаём три ВМ с Ubuntu 24.04
multipass launch --name primary
multipass launch --name standby
multipass launch --name client
```

2. Организация пространства для скриптов

На хосте Windows создаём папку C:\Users\AsusAspire\3\Desktop\MOUNT.

Создаем папки для каждого узла:

Client - C:\Users\AsusAspire\3\Desktop\MOUNT\mountpointCLIENT
Primary - C:\Users\AsusAspire\3\Desktop\MOUNT\mountpointPRIMARY
Standby - C:\Users\AsusAspire\3\Desktop\MOUNT\mountpointSTANDBY

Там будут находиться скрипты для каждого узла

Монтируем эти папки во все три ВМ

```
multipass mount " C:\Users\AsusAspire\3\Desktop\MOUNT\mountpointPRIMARY"
primary:/home/ubuntu/mountpoint
multipass mount " C:\Users\AsusAspire\3\Desktop\MOUNT\mountpointSTANDBY"
standby:/home/ubuntu/mountpoint
multipass mount " C:\Users\AsusAspire\3\Desktop\MOUNT\mountpointCLIENT"
client:/home/ubuntu/mountpoint
```

Из-за особенностей файловой Windows скрипты не получают права на исполнение внутри ВМ (chmod +x не работает). Поэтому все скрипты будем запускать через **bash**.

2.1. Файл env (общие переменные окружения)

В каждую из трёх папок (mountpointPRIMARY, mountpointSTANDBY, mountpointCLIENT) был помещён файл env со следующим содержимым:

```
#!/usr/bin/env bash

export PG_VER=16
export PGDATA="/var/lib/postgresql/${PG_VER}/main"

export PG_USER=postgres
export PG_PASS='postgres123'

export REPL_USER='replicator'
export REPL_PASS='replpass123'

export CLIENT_NET='0.0.0.0/0'

export PRIMARY_IP='172.30.219.223'
export STANDBY_IP='172.30.213.180'
export CLIENT_IP='172.30.213.250'
```

2.2. Файл common (вспомогательные функции)

В те же папки был добавлен файл common:

```
#!/usr/bin/env bash
set -euo pipefail

log() { echo "[INFO] $*"; }
ok() { echo "[OK] $*"; }
warn() { echo "[WARN] $*"; }
die() { echo "[ERR] $*" >&2; exit 1; }

ensure_line() {
    local file="$1"
    local line="$2"
    sudo grep -qxF "$line" "$file" || echo "$line" | sudo tee -a "$file" >/dev/null
}

ensure_block_once() {
    local file="$1"
    local marker="$2"
    shift 2
    local block="$*"
    if ! sudo grep -qF "$marker" "$file"; then
        printf '\n%s\n' "$block" | sudo tee -a "$file" >/dev/null
    fi
}
```

3. Настройка PostgreSQL (потокковая репликация)

После подготовки общей папки со скриптами были выполнены следующие действия.

3.1. На узле primary

Запущен скрипт `setup_primary`, который:

- устанавливает PostgreSQL;
- настраивает `postgresql.conf` для приёма подключений, репликации и WAL;
- добавляет правила в `pg_hba.conf` для репликатора и клиента;
- создаёт пользователя для репликации и устанавливает пароль для postgres.

Скрипт `setup_primary`:

```
#!/usr/bin/env bash
set -euo pipefail

source ./env
source ./common

log "Установка PostgreSQL"
sudo apt-get update -qq
sudo apt-get install -y -qq postgresql postgresql-client

log "Настройка postgresql.conf"
ensure_block_once "/etc/postgresql/${PG_VER}/main/postgresql.conf" "# pg-lab streaming replication" \
"listen_addresses = '*'
wal_level = replica
max_wal_senders = 5
wal_keep_size = 64
archive_mode = off"

log "Настройка pg_hba.conf"
ensure_line "/etc/postgresql/${PG_VER}/main/pg_hba.conf" "host replication ${REPL_USER}
${STANDBY_IP}/32 md5"
ensure_line "/etc/postgresql/${PG_VER}/main/pg_hba.conf" "host all          postgres  ${CLIENT_NET} md5"
ensure_line "/etc/postgresql/${PG_VER}/main/pg_hba.conf" "host all          postgres  ${STANDBY_IP}/32
md5"

log "Перезапуск PostgreSQL"
sudo systemctl restart postgresql

log "Создание пользователя репликации"
sudo -u postgres psql <<SQL
ALTER USER postgres PASSWORD '${PG_PASS}';

DO \$$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = '${REPL_USER}') THEN
    CREATE ROLE ${REPL_USER} WITH REPLICATION LOGIN PASSWORD '${REPL_PASS}';
  END IF;
END
\$$;
SQL
ok "Primary настроен"
```

3.2. На узле standby

Запущен скрипт `setup_standby`, который:

- устанавливает PostgreSQL;
- останавливает сервис, удаляет старую PGDATA;
- копирует данные с primary с помощью `pg_basebackup`;
- разрешает подключения для клиента;
- запускает PostgreSQL и проверяет режим восстановления.

Скрипт `setup_standby`:

```
#!/usr/bin/env bash
set -euo pipefail

source ./env
source ./common

log "Установка PostgreSQL"
sudo apt-get update -qq
sudo apt-get install -y -qq postgresql postgresql-client

log "Остановка сервиса перед basebackup"
sudo systemctl stop postgresql || true

log "Удаление старого data directory"
sudo rm -rf "${PGDATA}"

log "Клонирование с primary"
sudo -u postgres PGPASSWORD="${REPL_PASS}" pg_basebackup \
  -h "${PRIMARY_IP}" \
  -U "${REPL_USER}" \
  -D "${PGDATA}" \
  -P -Xs -R -w \
  --checkpoint=fast

log "Разрешаю подключения"
ensure_block_once "/etc/postgresql/${PG_VER}/main/postgresql.conf" "# pg-lab standby access" \
  "listen_addresses = '*'"

ensure_line "/etc/postgresql/${PG_VER}/main/pg_hba.conf" "host all postgres ${CLIENT_NET} md5"

log "Запуск PostgreSQL"
sudo systemctl start postgresql

log "Проверка режима восстановления"
sudo -u postgres psql -c "SELECT pg_is_in_recovery();"

ok "Standby настроен"
```

3.3. На узле client

Установлены клиентские утилиты PostgreSQL:

```
sudo apt-get update -qq  
sudo apt-get install -y -qq postgresql-client
```

После этого было выполнено подключение к primary для проверки доступности:

```
psql -h primary -U postgres -d postgres
```

3.4. Проверка репликации

На standby выполнены следующие запросы (через `sudo -u postgres psql`):

```
SELECT * FROM customers;  
SELECT * FROM products;  
SELECT pg_is_in_recovery();
```

Первые два запроса показывают данные, синхронизированные с primary (предварительно на primary были созданы таблицы customers и products с несколькими строками). Третий запрос возвращает true, подтверждая, что узел работает в режиме только чтения (реплика).

3.5. Демонстрация клиентских сессий

На узле client был запущен скрипт show_sessions, который:

- запускает две фоновые сессии к primary с длительным запросом (`pg_sleep(40)`);
- выводит список активных процессов (`pg_stat_activity`) на момент выполнения;
- дожидается завершения фоновых процессов.

Скрипт show_sessions:

```
#!/usr/bin/env bash  
set -euo pipefail  
  
source ./env  
source ./common  
  
log "Запускаю две длительные сессии"  
sudo bash -lc "  
PGPASSWORD='${PG_PASS}' psql -h '${PRIMARY_IP}' -U postgres -d postgres -c \"SELECT 'session1' AS s,  
pg_backend_pid(), pg_sleep(40);\" >/tmp/session1.log 2>&1 &  
PGPASSWORD='${PG_PASS}' psql -h '${PRIMARY_IP}' -U postgres -d postgres -c \"SELECT 'session2' AS s,  
pg_backend_pid(), pg_sleep(40);\" >/tmp/session2.log 2>&1 &  
sleep 2  
PGPASSWORD='${PG_PASS}' psql -h '${PRIMARY_IP}' -U postgres -d postgres -c \"SELECT pid, username,  
client_addr, state, query FROM pg_stat_activity WHERE datname='postgres' ORDER BY pid;\"  
wait  
"  
ok "Сессии показаны"
```


Этап 2. Симуляция и обработка сбоя.

1. Заполнение дискового пространства на primary

На узле primary был запущен скрипт `fill_disk`, который последовательно создаёт большие файлы в каталоге `pg_wal` (поддиректория `PGDATA`). Запись продолжается до тех пор, пока не будет исчерпано свободное место на разделе.

Скрипт `fill_disk` (primary):

```
#!/usr/bin/env bash
set -euo pipefail

source ./env
source ./common

log "Создание мусорных файлов"

i=1
while sudo dd if=/dev/zero of="${PGDATA}/pg_wal/fill_${i}.bin" bs=32M count=8 conv=fsync
status=progress; do
    i=$((i+1))
done

warn "Диск заполнен, показываю место"
df -h "$PGDATA" || true

warn "Последние строки лога PostgreSQL"
sudo tail -30 /var/log/postgresql/postgresql-${PG_VER}-main.log || true
```

Вывод (фрагмент):

```
[INFO] Создание мусорных файлов
8+0 records in
8+0 records out
268435456 bytes (268 MB, 256 MiB) copied, 0.196599 s, 1.4 GB/s
...
dd: error writing '/var/lib/postgresql/16/main/pg_wal/fill_7.bin': No space left on device
```

Скрипт остановился при попытке записать очередной файл, получив ошибку «No space left on device». Это симулирует аварийную ситуацию, когда на разделе с данными PostgreSQL заканчивается место.

2. Попытка записи данных на primary (клиент)

На узле client был выполнен скрипт fill_data, который запускает несколько транзакций на primary: вставка заказов, обновление статусов, изменение остатков товаров.

Скрипт fill_data (client):

```
#!/usr/bin/env bash
set -euo pipefail

source ./env
source ./common

log "Показываю несколько клиентских сессий"
sudo PGPASSWORD="${PG_PASS}" psql -h "${PRIMARY_IP}" -U postgres -d postgres -c
"SELECT pg_backend_pid(), now();"
sudo PGPASSWORD="${PG_PASS}" psql -h "${PRIMARY_IP}" -U postgres -d postgres -c
"SELECT pg_backend_pid(), now();"

log "Две транзакции на primary"
sudo env PGPASSWORD="${PG_PASS}" psql -h "${PRIMARY_IP}" -U postgres -d postgres
<<'SQL'
BEGIN;
INSERT INTO orders (customer_id, status)
SELECT id, 'new' FROM customers WHERE email = 'ivan@example.com';
INSERT INTO order_items (order_id, product_id, quantity, unit_price)
SELECT o.id, p.id, 1, p.price
FROM orders o
JOIN products p ON p.sku = 'NB-001'
WHERE o.customer_id = (SELECT id FROM customers WHERE email = 'ivan@example.com')
ORDER BY o.id DESC
LIMIT 1;
COMMIT;

BEGIN;
UPDATE orders
SET status = 'paid'
WHERE customer_id = (SELECT id FROM customers WHERE email = 'ivan@example.com');
UPDATE products
SET stock = stock - 1
WHERE sku = 'NB-001';
COMMIT;

SELECT c.full_name, o.id AS order_id, o.status, p.product_name, oi.quantity, oi.unit_price
FROM orders o
JOIN customers c ON c.id = o.customer_id
LEFT JOIN order_items oi ON oi.order_id = o.id
LEFT JOIN products p ON p.id = oi.product_id
ORDER BY o.id;
SQL
ok "Данные записаны"
```

Результат: выполнение транзакций привело к ошибке на primary из-за отсутствия места на диске. В логах PostgreSQL (и в выводе скрипта) появилось сообщение:

```
2026-04-12 22:04:37.783 +04 [4280] postgres@postgres ERROR: could not extend file
"base/5/16414": No space left on device
```

Таким образом, основной узел перестал принимать изменения.

3. Переключение (failover) на standby

На узле standby был выполнен скрипт promote, который переводит резервный сервер в режим чтения-записи (становится новым primary).

Скрипт promote (standby):

```
#!/usr/bin/env bash
set -euo pipefail

source ./env
source ./common

log "Промоут standby в primary"
sudo -u postgres psql -c "SELECT pg_promote();"
sleep 2

log "Проверка статуса"
sudo -u postgres psql -c "SELECT pg_is_in_recovery();"

ok "Standby стал новым primary"
```

Вывод:

```
[INFO] Промоут standby в primary
pg_promote
-----
t
(1 row)

[INFO] Проверка статуса
pg_is_in_recovery
-----
f
(1 row)

[OK] Standby стал новым primary
```

Значение f (false) для pg_is_in_recovery() подтверждает, что узел больше не находится в режиме восстановления и принимает запросы на запись.

4. Подготовка standby для будущего восстановления primary

Чтобы впоследствии старый primary мог быть перестроен как реплика нового мастера, на standby (теперь активный primary) был выполнен скрипт `allow_reclone`. Он добавляет в `pg_hba.conf` правило, разрешающее подключение пользователя репликации с IP-адреса старого primary.

Скрипт `allow_reclone` (standby):

```
#!/usr/bin/env bash
set -euo pipefail

source ./env
source ./common

ensure_line "/etc/postgresql/${PG_VER}/main/pg_hba.conf" "host replication ${REPL_USER}
${PRIMARY_IP}/32 md5"
ensure_line "/etc/postgresql/${PG_VER}/main/pg_hba.conf" "host all          postgres
${CLIENT_NET} md5"

sudo systemctl reload postgresql || true

ok "Разрешение для повторного клонирования добавлено"
```

После этого старый primary сможет выполнить `pg_basebackup` от нового мастера.

5. Восстановление основного узла

После переключения (failover) на standby (новый primary) клиентские подключения были перенаправлены на этот узел. Например:

```
psql -h standby -U postgres -d postgres
```

Узел standby теперь принимает запросы на запись и является активным primary.

5.1. Восстановление старого primary как реплики

На узле primary (старый мастер, который вышел из строя из-за переполнения диска) был запущен скрипт `recover_primary`. Он:

- останавливает PostgreSQL;
- полностью удаляет старую PGDATA;
- выполняет `pg_basebackup` с нового primary (узла standby);
- настраивает права доступа к файлам данных;
- перезапускает PostgreSQL и проверяет, что узел находится в режиме восстановления (только чтение).

Скрипт recover_primary (primary):

```
#!/usr/bin/env bash
set -euo pipefail

source ./env
source ./common

log "Остановка PostgreSQL на старом primary"
sudo systemctl stop postgresql || true

log "Удаление старого data directory"
sudo rm -rf "${PGDATA}"

log "Создание пустого каталога"
sudo mkdir -p "${PGDATA}"
sudo chown postgres:postgres "${PGDATA}"
sudo chmod 700 "${PGDATA}"

log "Клонирование данных с нового primary"
sudo -u postgres bash -c "
export PGPASSWORD='${REPL_PASS}'
pg_basebackup \
  -h '${STANDBY_IP}' \
  -U '${REPL_USER}' \
  -D '${PGDATA}' \
  -P -Xs -R -w \
  --checkpoint=fast
"

# ВАЖНО: права после pg_basebackup
log "Исправляем права на PGDATA"
sudo chown -R postgres:postgres "${PGDATA}"
sudo find "${PGDATA}" -type d -exec chmod 700 {} \;
sudo find "${PGDATA}" -type f -exec chmod 600 {} \;
sudo chmod 700 "${PGDATA}/pg_wal" || true

log "Проверка primary_conninfo"
sudo grep primary_conninfo "${PGDATA}/postgresql.auto.conf" || true

ensure_block_once "/etc/postgresql/${PG_VER}/main/postgresql.conf" "# pg-lab standby access" \
  "listen_addresses = '*'
  max_wal_senders = 10"

ensure_line "/etc/postgresql/${PG_VER}/main/pg_hba.conf" \
  "host all postgres ${CLIENT_NET} md5"

log "Запуск PostgreSQL"
sudo systemctl start postgresql

sleep 3

log "Проверка, что узел стал standby"
sudo -u postgres psql -c "SELECT pg_is_in_recovery();"

ok "Старый primary восстановлен как standby"
```

После завершения скрипта старый primary снова работает как реплика (hot standby) и синхронизирует данные с новым мастером (standby).

5.2. Проверка статуса репликации

На новом primary (узле standby) была выполнена команда для просмотра активных реплик:

```
sudo -u postgres psql -c "select * from pg_stat_replication;"
```

5.3. Проверка целостности данных

На обоих узлах (standby и primary) были выполнены одинаковые запросы для подсчёта строк в таблицах. Результаты должны совпадать.

```
select count(*) from customers;  
select count(*) from products;
```

Выполнение (на любом из узлов):

```
postgres=# select count(*) from customers;  
count  
-----  
      3  
(1 row)  
  
postgres=# select count(*) from products;  
count  
-----  
      4  
(1 row)
```

Это подтверждает, что все изменения, сделанные на новом primary в момент сбоя, были переданы на восстановленный узел, и кластер снова работает в исходной конфигурации (один мастер для записи, один горячий резерв).

Вывод

В ходе выполнения работы на двух виртуальных машинах была настроена потоковая репликация PostgreSQL. При симуляции сбоя (заполнение дискового раздела основного узла «мусорными» файлами) выполнен аварийный переключатель на резервный сервер с сохранением работоспособности клиентов. После устранения причины сбоя основной узел восстановлен и повторно синхронизирован. Исходная конфигурация отказоустойчивого кластера полностью восстановлена. Получены практические навыки настройки репликации, обработки отказа и восстановления PostgreSQL-кластера.